算法设计与分析作业三

xxxxxxxxx xxx

2026 年 3 月 25 日

题 1 (4.3)**.**

解答. 主要设计思想:

本题可以转化为一个区间覆盖问题。每个基站可以覆盖左右各 4 千米的范围，也就是说，一个基站最长可以覆盖长度为 8 千米的路段。

采用贪心策略：从左向右扫描每所房子。对于第一所没有被覆盖的房子 d_i，为了使基站向右覆盖更多的后续房子，我们应该将基站直接建在它的右侧最远有效距离处，即基站的位置设在 $pos = d_i + 4$ 处。这样该基站的覆盖范围就是 $[d_i, d_i + 8]$。接着，我们跳过所有在这座基站覆盖范围内的房子（即满足 $d_k \leq pos + 4$ 或 $d_k \leq d_i + 8$ 的房子），遇到下一个未被覆盖的房子时重复上述过程，直到所有房子都被覆盖。由于房子距离已经升序给出（$d_1 < d_2 < \cdots < d_n$），只需要线性扫描一遍即可。

伪代码描述:

Algorithm 1: 基站位置选择算法

Input: 房子到 A 点的距离数组 $D = [d_1, d_2, \ldots, d_n]$ $(d_1 < d_2 < \cdots < d_n)$

Output: 最少基站的数量 $count$ 和各个基站的位置集合 S

```
count ← 0;
S ← ∅;
i ← 1;
while i ≤ n do
    // 将基站建在当前房子右侧最远有效处
    pos ← D[i] + 4;
    S ← S ∪ {pos};
    count ← count + 1;
    // 寻找下一个未被这个基站覆盖的房子
    while i ≤ n and D[i] ≤ pos + 4 do
        i ← i + 1;
return count, S;
```

正确性证明:

使用**贪心选择性质**（替换法）证明：假设存在一个最优解 O（按位置从左到右排序），其放置的第一个基站位置为 b_1。为了覆盖最左侧的房子 d_1，必须满足 $d_1 \geq b_1 - 4$，即 $b_1 \leq d_1 + 4$。

我们的贪心算法选择在 $pos_1 = d_1 + 4$ 处放置第一个基站。由于 O 是一组合法的解，有 $b_1 \le d_1 + 4 = pos_1$。这意味着，贪心选择的基站 pos_1 向右延伸的覆盖终点 $pos_1 + 4 = d_1 + 8$ 必然大于等于 O 中首个基站的向右覆盖终点 $b_1 + 4$。同时，由于最左侧房子就是 d_1（没有比 d_1 更靠左的房子），无论基站建在 b_1 还是 pos_1，左侧所有的房子都被覆盖了。

因此，如果在最优解 O 中，我们将第一个基站的位置从 b_1 移动到 $pos_1 = d_1 + 4$ 处，它依然能够覆盖原来 b_1 覆盖的所有房子，甚至可能向右覆盖更多的新房子。替换后并不增加基站总数，所有的房子依然被覆盖，所以含有这个贪心选择的解依然是一个最优解。

放置第一个基站后，除去被覆盖的房子，剩下的房子依然保持单调递增，构成了与原问题形式相同且规模更小的子问题，具备**最优子结构**性质。通过归纳可知，每次都做贪心选择最终必定会产生全局最优解。

最坏情况时间复杂度：

尽管算法中有内外两层循环，但外层 ‘while‘ 控制基站建立，内层 ‘while‘ 用来跳过被覆盖的房子。变量 i 从 1 开始只增不减直到跨越 n。因此，每所房子的位置只会被访问恰好一次。整个数组扫描一遍的时间复杂度为最坏 $\mathcal{O}(n)$ 级别。 □

题 2 (4.6)**.**

解答. **算法设计：**

本题是一个经典的调度问题（Shortest Job First, 短作业优先）。要使得平均完成时间最少，即最小化 $\sum_{i=1}^{n} w(i)$，我们应当将加工时间 $t(i)$ 越短的作业安排在越前面执行。

具体算法如下：

1. 将所有作业按照其加工时间 $t(i)$ 从小到大进行升序排序。
2. 设排序后的作业序列为 $J' = \{j_1, j_2, \ldots, j_n\}$，满足 $t(j_1) \le t(j_2) \le \cdots \le t(j_n)$。
3. 依次安排作业：
   - 第 1 个作业 j_1 的开始时间 $f(j_1) = 0$，完成时间 $w(j_1) = t(j_1)$；
   - 对于第 k 个作业 j_k $(2 \le k \le n)$，其开始时间为前一个作业的完成时间，即 $f(j_k) = w(j_{k-1})$，完成时间为 $w(j_k) = f(j_k) + t(j_k)$。

正确性证明：

我们可以使用**交换论证法**来证明该贪心策略的最优性。

假设我们有一个排好的作业序列，其总完成时间为 W。在这个序列中，如果存在两个相邻的作业 A 和 B（A 在 B 之前执行），且满足 $t(A) > t(B)$（即发生了逆序，不符合我们“短作业优先”的贪心策略）。

设定作业 A 的开始时间为 T。

- 在交换前：作业 A 的完成时间为 $w(A) = T + t(A)$；作业 B 的开始时间为 $T + t(A)$，完成时间为 $w(B) = T + t(A) + t(B)$。这两项作对总完成时间的贡献为 $w(A) + w(B) = 2T + 2t(A) + t(B)$。

- 我们现在将作业 A 和 B 的执行顺序交换（B 先执行，A 后执行）：作业 B 的完成时间为 $w'(B) = T + t(B)$；作业 A 的完成时间为 $w'(A) = T + t(B) + t(A)$。交换后这两项对总完成时间的贡献为 $w'(A) + w'(B) = 2T + 2t(B) + t(A)$。

让我们计算交换前后总完成时间的差值：

$$[w(A) + w(B)] - [w'(A) + w'(B)] = [2T + 2t(A) + t(B)] - [2T + 2t(B) + t(A)] = t(A) - t(B)$$

因为已知 $t(A) > t(B)$，所以差值 $t(A) - t(B) > 0$。这意味着交换后的总完成时间变小了！

并且，由于 A 和 B 的交换仅改变了它们两者的完成时间，它们合并起来的总时间 $T + t(A) + t(B)$ 并没有改变，因此不会影响排在它们之后其他任何作业的开始和完成时间。

由此可以得出结论：在任何给定的调度序列中，只要存在加工时间长的作业排在加工时间短的作业前面，我们就可以通过交换它们来严格减少（或不改变）总完成时间。当且仅当所有作业按照加工时间从小到大排列时，总完成时间（以及平均完成时间）达到最小。这证明了短作业优先（SJF）算法的正确性。

最坏情况时间复杂度：

算法的核心步骤是对 n 项作业的加工时间进行排序，使用诸如快速排序或归并排序的高效排序算法，时间复杂度为 $O(n \log n)$。排序后计算安排时间只需单次遍历数组，耗时 $O(n)$。因此，总体最坏情况下的时间复杂度为 $O(n \log n)$。 □

题 3 (4.13)**.**

解答. **算法设计：**

由于每项作业的加工时间均为 1，在相同的截止时间 D 之前，这台机器最多只能完成 D 项作业。被安排在时刻 D 及其之后开始的作业都会在 D 之后完成，从而产生罚款。为了使总罚款最少，我们应该尽可能让罚款 $m(i)$ 较大的作业在 D 之前完成。

基于此，可以设计如下贪心算法：

1. 若 $n \leq D$，则所有作业都可以在截止时间前完成，不需要交任何罚款。随意将所有作业安排在 0 到 $n-1$ 时刻开始即可。

2. 若 $n > D$，将所有的 n 项作业按照其对应的罚款金额 $m(i)$ 从大到小进行降序排序。

3. 选取排序后的前 D 项作业，将它们分别安排在时刻 $0, 1, 2, \ldots, D-1$ 开始加工（这些作业能够按时完成，不被延误）。

4. 对于剩余的 $n-D$ 项作业，将它们安排在时刻 $D, D+1, \ldots, n-1$ 开始加工（这些作业会被延误，总罚款为它们的 $m(i)$ 之和）。

正确性证明：

我们需要最小化被延误作业的总罚款 $\sum_{j \in \text{延误}} m(j)$。由于所有作业的总罚款代价 $M = \sum_{i=1}^{n} m(i)$ 是一个固定的常数，这等价于我们要**最大化按时完成的作业的总罚款** $\sum_{k \in \text{按时}} m(k)$。

因为每个作业耗时均为 1，且截止时间都为 D。区间 $[0, D]$ 总时长为 D，所以任何一种合法的调度方案最多只能安排 $\min(n, D)$ 个作业按时完成。

假设存在一个最优调度 O，其按时完成的作业集合为 A，且 $|A| \leq D$。如果我们用贪心策略选出的前 D 大罚款作业的集合为 B：如果 $A \neq B$，那么必然存在一个未被 O 安排按时完成的作业 $x \in B$ 且 $x \notin A$，以及一个被 O 安排按时完成的作业 $y \in A$ 且 $y \notin B$。

由于 B 中包含了全集里最大的 D 个罚款，而 $x \in B$ 但 $y \notin B$，且各项罚款彼此不等，必然有 $m(x) > m(y)$。我们可以在最优调度 O 中，将按时完成部分的作业 y 替换为作业 x（交换它们的调度时间）。替换后，按时完成的作业罚款总和的变化量为 $m(x) - m(y) > 0$。这意味着按时完成的罚款增加了，因而延误作业的总罚款减小了！这就与 O 是最优调度的假设产生了矛盾。

因此，使得总罚款最少的唯一最优调度，必须是选择罚款 $m(i)$ 最大的前 D 个作业按时完成，其余作业延误。这就证明了该贪心策略的正确性。

最坏情况时间复杂度：

该算法的主要开销在于按罚款金额 $m(i)$ 对所有作业进行降序排序。采用高效的比较排序算法（如快速排序或归并排序），排序的时间复杂度为 $O(n \log n)$。排序后分配具体的加工时间只需遍历一次作业序列即可，耗时 $O(n)$。因此，该算法在最坏情况下的整体时间复杂度为 $O(n \log n)$。 □

题 4 (4.18)**.**

解答. **(1) 所有广告效益相等的情况**

此时问题转化为传统的"区间调度问题"，目标是选取数量最多的互不重叠的广告区间。

算法设计：

1. 已经已知广告按截止时间 $d(1) \leq d(2) \leq \cdots \leq d(n)$ 排好序，无需再次排序。
2. 初始化已选集合 $A = \{1\}$，记录当前最后完成的广告 $last = 1$。
3. 从 $i = 2$ 遍历到 n，如果第 i 个广告的开始时间 $s(i) \geq d(last)$（说明与当前已选的最后广告不冲突），则将其加入集合 A，并更新 $last = i$。
4. 遍历结束后返回 A 的大小即为最大收益。

正确性证明（贪心选择性质）：假设最优解 O 选出的广告按截止时间排序为 $\{j_1, j_2, \ldots, j_k\}$。贪心算法选出的第一个广告是 1。如果 $j_1 \neq 1$，由于广告组已按截止时间升序排列，必然有 $d(1) \leq d(j_1)$。我们可以用广告 1 替换最优解中的 j_1。由于 $d(1) \leq d(j_1)$，广告 1 的结束时间比 j_1 更早或相同，它绝不会与 j_2 发生重叠（因为 $s(j_2) \geq d(j_1) \geq d(1)$）。替换后集合的广告数量不减，依然是一个最优解。通过归纳法可以得出，每次选择最早结束的兼容广告，最终能获得最大数量的广告组合。

时间复杂度：因为数组已经按截止时间排好序，算法只需对数组进行一次遍历，因此时间复杂度为 $O(n)$。如果未经排序，需要先用 $O(n \log n)$ 排序。

(2) 广告效益可以取任意正整数的情况

此时贪心算法不再适用（比如某个长区间广告虽然结束时间晚，但它带来的效益极大，可能远超许多个小区间广告），需要使用**动态规划**解决。这是一个"带权区间调度问题"。

算法设计思想：

1. 同样假设广告已经按截止时间 $d(1) \leq d(2) \leq \cdots \leq d(n)$ 排好序。

2. 定义一个数组 $p[i]$：表示在广告 i 之前结束，并且与广告 i 不重叠（相容）的最晚广告的编号（即寻找最大的 $j < i$ 使得 $d(j) \leq s(i)$）。如果不存在这样的广告，则让 $p[i] = 0$。为了高效查找，可结合二分查找预处理出 p 。

3. 定义 $dp[i]$ 为只考虑前 i 个广告时能够拿到的最大总效益。初始 $dp[0] = 0$。

4. 状态转移：考虑第 i 个广告，我们有两种选择：

   - **不选**广告 i，此时的最大效益等于前 $i-1$ 个广告的最大效益，即 $dp[i-1]$。
   - **选择**广告 i，此时效益等于广告 i 的效益 $v(i)$ 加上与它相容的、在它之前的所有广告中的最大效益，即 $v(i) + dp[p[i]]$。

   选取两者的最大值作为状态转移方程：

$$dp[i] = \max(dp[i-1],\ v(i) + dp[p[i]])$$

5. 从 1 到 n 依次计算 $dp[i]$，最终 $dp[n]$ 即为所求的最大总效益。

最坏情况时间复杂度：计算并预处理 p 数组时，对于每个广告 i，通过二分查找在 1 到 $i-1$ 范围内定位其最近的相容广告，单次二分需 $O(\log n)$，整体预处理的时间复杂度为 $O(n \log n)$。动态规划的递推过程只需要一次从前向后的线性遍历，时间复杂度为 $O(n)$。所以，该算法的整体最坏情况下的时间复杂度为 $O(n \log n)$。 □

题 5 (5.2)**.**

解答. 这是一个带有约束限制的组合优化问题，可以使用**回溯法 (Backtracking)** 来系统地搜索解空间树，寻找在总价格不超过 120 的前提下达到总重量最轻的组合。在这类问题中，通过剪枝可以在不漏掉最优解的前提下大幅减少计算量。

状态空间与变量定义：

- 我们需要为 4 种零件各自选择一个供应商，i 表示当前决定第 i 个零件的供应商，范围为 $1 \leq i \leq 4$。
- 分支：每个零件都有 3 个供应商可选，所以每一层展开 3 个分支（$j \in \{1, 2, 3\}$）。
- 设 $C[i][j]$ 为第 i 种零件在第 j 个供应商处的 ** 价格 **，$W[i][j]$ 为 ** 重量 **。
- $current_cost$：当前已选择零件的总价格。
- $current_weight$：当前已选择零件的总重量。
- min_weight：整个搜索过程中发现的在预算范围内的最轻重量记录（初始化为无穷大 ∞ 或者是极大值）。

约束函数与剪枝条件：在决定选择零件 i 由供应商 j 提供时：

1. **价格上限约束（可行性剪枝）**：如果加上该零件的价格后，总价格超过了预算上限 120，即 $current_cost + C[i][j] > 120$，那么这条分支及往下探索的所有组合必然超预算，直接剪枝返回。

2. **重量下界约束（最优性剪枝）**：如果加上该零件的重量后，当前总重量已经大于等于已知的最优解 min_weight，即 $current_weight + W[i][j] \geq min_weight$，那么沿着这条道路走下去，就算后面所有物品重量是 0 也不可能打破纪录，因此直接剪枝返回。

回溯算法伪代码：

Algorithm 2: 最小重量机器设计 - 回溯搜索

```
Input: 零件种类 n = 4，供应商种类 m = 3，最大预算 MaxCost = 120，价格矩阵
       C，重量矩阵 W
Output: 满足预算情况下的最小重量组合的重量 min_weight
min_weight ← ∞;
best_choice ← 空数组 ;                          // 记录最优的供应商组合
current_choice ← 全0 数组;
Function Backtrack(i, current_cost, current_weight):
    // 如果已经选完了所有的零件，且没有被剪枝，更新最优记录
    if i > n then
        if current_weight < min_weight then
            min_weight ← current_weight;
            best_choice ← current_choice;
        return;
    // 枚举第 i 种零件的所有可能供应商
    for j ← 1 to m do
        // 可行性剪枝：预算限制
        if current_cost + C[i][j] > MaxCost then
            continue;
        // 最优性剪枝：已经比目前最好的记录重了，没必要继续
        if current_weight + W[i][j] ≥ min_weight then
            continue;
        // 做出选择进入下一层递归
        current_choice[i] ← j;
        Backtrack(i + 1, current_cost + C[i][j], current_weight + W[i][j]);
调用 Backtrack(1, 0, 0);
return min_weight;
```

□

题 6 (5.5)**.** 给出 *8* 皇后问题的一个广度优先回溯算法，并分析该算法的时间复杂度。

解答. 本题要求回答的是 **8 皇后问题的一个广度优先（也就是分支限界法中的队列式分支限界）回溯算法**，并且分析其时间复杂度。

1. 广度优先回溯（分支限界）算法设计

广度优先搜索通常借助**队列(Queue)**来实现。对于 8 皇后问题(或拓展到 n 皇后问题),我们将棋盘的每一行看作状态树的一层。状态可以用一个一维数组(或列表)表示,例如 $[c_1, c_2, \ldots, c_k]$ 表示前 k 行中，第 i 行的皇后放置在第 c_i 列。

算法步骤与伪代码:

Algorithm 3: 8 皇后问题的广度优先回溯算法

```
Input: 棋盘大小 n = 8
Output: 所有的可行解
初始化一个空队列 Q;
将初始状态（空列表 []，表示放置了 0 个皇后）入队 Q;
while Q 不为空 do
    从 Q 中出队一个状态 S;
    记此时 S 中已经放置了 k 个皇后（即 S 的长度为 k）;
    if k == n then
        输出状态 S 作为一个可行解;
        continue;
    // 考察下一行（第 k+1 行）的所有可能列
    for col ← 1 to n do
        if 在第 k+1 行、第 col 列放置皇后不与 S 中已有的 k 个皇后冲突 then
            // 冲突检测条件：同一列（S[i] == col）或同一对角线
               (|S[i] − col| == |i − (k+1)|)
            生成新的状态 S' = S 合并 col;
            将 S' 入队 Q;
```

冲突检测的具体逻辑：对于欲在 $(k+1, col)$ 放置新皇后，遍历已有的 k 个皇后 $(i, S[i])$ $(1 \le i \le k)$。若满足以下全部条件，则不冲突：

1. 不在同一列：$S[i] \neq col$
2. 不在主副对角线上：$|k+1-i| \neq |col - S[i]|$

2. 算法的时间复杂度分析

- **节点扩展空间（状态空间树大小）**：在 8 皇后（或普通化为 n 皇后）问题中，每一行必须放置一个皇后。如果不加任何剪枝，第一行有 n 种选法，第二行有 n 种选法……总共是 n^n 个节点。但由于算法中显式地排除了同列和同对角线的情况，实际搜索生成的树节点数会大大减少。在最坏的理论上界分析中，考虑到同行同列的互斥，它的排列上限是排列数 $n!$。
- **节点处理复杂度**：对于每个从队列弹出的状态结点，我们要循环考察 n 个列。对于每个列，都要与前面的 k 级状态进行冲突判断，单次检查时间最坏为 $O(k) \le O(n)$。

- **整体时间复杂度**：扩展节点数的上界为 $O(n!)$。在每个节点，最多进行 n 次探测，每次探测需要 $O(n)$ 时间。因此总体的时间复杂度上界为 $O(n^2 \cdot n!)$。也有分析将其记为 $O(n!)$ 级别（因为节点分支数随深度下降极快，实际复杂度远达不到理论上界）。对于 $n = 8$ 这个常数而言，状态空间非常小（只有 92 个解），实际运行时间是常数阶 $O(1)$，但作为算法分析，应当写它的渐进复杂度即 $O(n!)$ 级别。

空间复杂度（补充）：由于是广度优先搜索，队列中将保存同一层的所有合法状态。对于 n 皇后问题，树的最大宽度可能是指数级的，因此其空间复杂度远高于深度优先搜索（DFS 空间为 $O(n)$），最坏情况下的空间复杂度亦为 $O(n!)$ 级别。 □

题 7 (5.6)**.** 子集和问题。设 n 个不同的正数构成集合 S ，求出使得和为某数 M 的 S 的所有子集。

解答. 本题是经典的"子集和问题"(Subset Sum Problem)，可以通过**回溯法**来搜寻所有可能的子集，并利用题目给出的一些特性进行**剪枝**以提高算法效率。

1. 算法设计思想

设集合 $S = \{s_1, s_2, \ldots, s_n\}$，其中的元素均为不同的正数。我们可以通过构造一棵子集树（二叉树 / 状态空间树）来遍历所有可能的组合，树的每一层代表是否选择当前元素进入子集：

- 对于第 i 层，有两个分支：选择将 s_i 加入当前子集，或者不将 s_i 加入当前子集。

状态和辅助变量定义：

- $current_sum$：当前已选择元素的和。
- $remain_sum$：未被考虑的剩余元素的总和（即 $\sum_{k=i}^{n} s_k$）。
- $path$：当前已经选择的元素的集合（可以通过数组或列表来动态记录）。

剪枝策略：为了避免不必要的盲目搜索，我们可以在进入深层递归前进行判断：

1. **超界剪枝**：因为集合中所有元素都是**正数**，如果在某一层状态下 $current_sum > M$，继续添加正数必然无法使和降到 M。因此可以直接剪枝。更优的做法是在初始阶段将集合 S 从小到大排序，这样当我们发现加上下一个元素已经超过 M 时，可直接停止当前分支。
2. **不足剪枝**：如果在某一阶段，已选和 $current_sum$ 加上所有剩余未处理元素的总和 $remain_sum$ 仍然小于 M（即 $current_sum + remain_sum < M$），那么即使把剩下的元素全选上也不可能达到 M。此时可以直接剪枝返回。

2. 算法伪代码

Algorithm 4: 子集和问题的回溯搜索算法

Input: 正数集合数组 S，目标和 M，元素个数 n

Output: 所有满足和为 M 的子集

```
对 S 进行升序排序（可选，为了更好的超界剪枝效果）;
remain_sum ← Σ_{i=1}^{n} S[i];
path ← ∅;
Function Backtrack(i, current_sum, remain_sum):
    // 找到一个可行解
    if current_sum == M then
        输出当前 path 作为一个解;
        return;
    // 达到叶节点，或者没找到
    if i > n then
        return;
    // 剪枝条件二：剩余元素全部选上也不够目标值
    if current_sum + remain_sum < M then
        return;
    // 分支一：选择加入当前元素 S[i]
    if current_sum + S[i] ≤ M then
        ;                                   // 剪枝条件一：保证加入后不超界
        path ← path ∪ {S[i]};
        Backtrack(i + 1, current_sum + S[i], remain_sum − S[i]);
        path ← path \ {S[i]} ;              // 回溯，撤销选择
    // 分支二：不加入当前元素 S[i]
    Backtrack(i + 1, current_sum, remain_sum − S[i]);
调用 Backtrack(1, 0, remain_sum);
```

3. 算法复杂度分析

- **时间复杂度**：在没有任何约束和剪枝的最坏情况下，算法需要遍历完整的子集树，包含 2^n 个叶子节点，时间复杂度为 $O(2^n)$。由于我们引入了有效的剪枝（排序预处理的时间为 $O(n \log n)$，与指数级开销相比可忽略），能够极大地剔除不符合条件的分支，使得实际运行时间通常远低于理论上的最坏上界。

- **空间复杂度**：递归的深度最大为 n（产生大小为 $O(n)$ 的调用栈空间）；记录当前状态的 $path$ 结构其长度亦不超过 n。因此，算法所需的额外空间复杂度主要为 $O(n)$。

□

题 8 (5.7)**.** 分派问题。给 n 个人分配 n 件工作，给第 i 个人分配第 j 件工作的成本是 $C(i,j)$。试求成本最小的工作分配方案。

解答. 本题是经典的带权二分图最佳匹配问题，由于这是一道在带有回溯和分支限界章节的作业中出现的问题，我们采用**回溯法（带有优化的分支限界思想）**对其进行设计和分析。如果不限定算法的话，该问题也可以用动态规划（状态压缩）或匈牙利算法的拓展（KM 算法）在多项式时间内完美解决。

1．算法设计思想（回溯与剪枝）

我们将寻找工作分配方案抽象为遍历一棵高度为 n 的排列树。每一层代表一个人，第 i 层表示为第 i 个人分配工作。

- 第 i 层有尚未被前 $i-1$ 个人选择的工作作为分支。

变量定义：

- $current_cost$：当前已经分配的人员累积的工作成本。
- min_cost：全局记录的当前最低总分配成本（可初始化为 ∞）。
- $visited[j]$：布尔数组，记录第 j 件工作是否已经被分配。

剪枝策略：为了降低排列树 $O(n!)$ 的探索开销，需要引入强大的剪枝条件。

1. **可行性剪枝（已超当前最优解）**：在第 i 层尝试分配工作给第 i 个人时，如果在还没有分配完所有人的情况下，$current_cost$ 已经大于或等于已知的全局最小成本 min_cost，那么沿着此分支持续走下去一定不会发掘出更优的方案。因此可以立刻截断此分支退回（剪枝）。
2. **启发式最优性剪枝(下界估计剪枝)**:为了让剪枝更提前生效,我们可以预先为每个人计算他所能接手的工作的最小可能成本(即矩阵每一行的最小值):$min_row[i] = \min_{1\le j\le n} C(i,j)$。如果在第 i 层时，我们不仅知道当前的已消费成本 $current_cost$，还能预估出接下来剩下的人哪怕所有人都取完美理想条件下的最小成本，这两部分相加也超出了 min_cost（即 $current_cost + \sum_{k=i+1}^{n} min_row[k] \ge min_cost$），就可以立即剪枝。

2．算法伪代码

Algorithm 5: 最小成本分派问题的分支限界回溯算法

```
Input: 人员数量 n，成本矩阵 C_{n×n}
Output: 最小的总成本 min_cost
min_cost ← ∞;
当前分配记录 path ← 空数组;
最佳分配记录 best_path ← 空数组;
visited ← 长为 n 的全 False 数组;
预处理每行最小值：min_row[i] ← min_j C[i][j], ∀i ∈ {1, ..., n};
计算后缀最小成本和表：tail_min_sum[i] = Σ_{k=i}^{n} min_row[k];
Function Backtrack(i, current_cost):
    // 如果所有人都分配完了
    if i > n then
        if current_cost < min_cost then
            min_cost ← current_cost;
            best_path ← path;
        return;
    // 下界启发式最优性剪枝
    if current_cost + tail_min_sum[i] ≥ min_cost then
        return;
    // 尝试将未分配的工作 j 分给第 i 个人
    for j ← 1 to n do
        if not visited[j] then
            visited[j] ← True;
            将 j 加入 path;
            Backtrack(i + 1, current_cost + C[i][j]);
            从 path 中移除 j ;                                  // 回溯
            visited[j] ← False ;                                // 回溯
调用 Backtrack(1, 0);
return min_cost, best_path;
```

3. 算法复杂度分析

- **时间复杂度**：在没有任何剪枝的最坏情况下，每个人都需要从剩下的工作中选取，总共的组合方案数为 $n!$。时间复杂度为 $O(n!)$。虽然使用下界启发式剪枝能够在绝大多数实际数据中将时间常数大幅压低，但在最坏情况下（例如 C 矩阵完全一致，无信息可利用）其复杂度上界依然是 $O(n!)$ 级别的。*注：如果利用匈牙利算法 (Kuhn-Munkres 算法) 求解二分图最大/小权匹配的话，时间复杂度可以降为多项式级别 $O(n^3)$。如果用状压 DP 的话，时间复杂度是 $O(n \cdot 2^n)$。*

- **空间复杂度**：递归维护的栈空间最大为 $O(n)$。此外，*visited*、*min_row* 等辅助数组也需

消耗 $O(n)$ 空间。所以本算法空间复杂度较优，为 $O(n)$。

$\square$